



CINEMAX

MANUALE TECNICO

LABORATORIO INTERDISCIPLINARE A -
YASEMIN KARAKUS
MAHA ILYAS CHOUDHRY

INDICE

1. Panoramica generale.....	3
2. Struttura del progetto	3
3. Come avviare l'applicazione.....	5
Prerequisiti	5
Compilazione	5
Avvio	5
Primo avvio.....	5
4. Ruoli utente e funzionalità per ruolo	5
6. Struttura del codice — spiegazione dei package	6
7. I Modelli (Models).....	7
Film.java.....	7
Proiezione.java	7
Prenotazione.java	8
Utente.java	8
Ruolo.java	9
8. I Manager (Managers).....	9
UtenteManager.java	9
ProiezioneManager.java	13
PrenotazioneManager.java	17
9. Gli Handler (Handlers).....	21
AuthHandler.java	21
GuestMenuHandler.java	21
ClienteMenuHandler.java	22
BigliettaioMenuHandler.java	22
ProiezionistaMenuHandler.java	23
10. Gli Helper (Helpers).....	24
CSVReader.java	24
ColoreConsole.java	25
RicercaProiezioneConsoleHelper.java	25
11. I dati — file CSV	26
data/utenti.csv.....	26
data/proiezioni.csv	26
data/prenotazioni.csv (generato automaticamente)	27
12. Sicurezza.....	27
Password.....	27
Validazioni implementate.....	27

1. Panoramica generale

CineMax è un sistema di gestione per un cinema monosala da 200 posti eseguito interamente da riga di comando. Consente di:

- Registrare e autenticare utenti con ruoli diversi
- In base al ruolo di un utente, consente di:
 - Cercare proiezioni cinematografiche
 - Prenotare biglietti per una proiezione
 - Gestire il programma delle proiezioni (palinsesto)
 - Visualizzare le prenotazioni

I dati sono salvati in file CSV.

2. Struttura del progetto

CineMax/

```
├── data/           ← file di dati (CSV)
|   ├── utenti.csv  ← elenco utenti registrati
|   └── proiezioni.csv ← elenco proiezioni programmate
|
└── src/
    ├── cinemax/
    |   ├── CineMax.java ← punto di avvio del programma
    |   |
    |   ├── Models/      ← classi che rappresentano i dati
    |   |   ├── Cinema.java
    |   |   ├── Film.java
    |   |   └── Proiezione.java
```

- | |— Prenotazione.java
- | |— Utente.java
- | |— Ruolo.java
- |
- |— Managers/ ← logica di business (operazioni sui dati)
- | |— UtenteManager.java
- | |— ProiezioneManager.java
- | |— PrenotazioneManager.java
- |
- |— Handlers/ ← gestione dei menu a schermo
- | |— AuthHandler.java
- | |— GuestMenuHandler.java
- | |— ClienteMenuHandler.java
- | |— BigliettaioMenuHandler.java
- | |— ProiezionistaMenuHandler.java
- |
- |— Helpers/ ← utilità di supporto
- | |— ColoreConsole.java
- | |— CSVReader.java
- | |— RicercaProiezioneConsoleHelper.java
- |
- |— ViewModels/ ← oggetti di trasporto dati
- | |— CriteriRicercaProiezione.java

3. Come avviare l'applicazione

Prerequisiti

- Java JDK 11 o superiore installato
- Un terminale (Command Prompt, PowerShell, o terminale di VS Code)

Compilazione

Dalla cartella radice del progetto (CineMax/), eseguire:

```
javac -d out src/cinemax/**/*.* src/cinemax/*.java
```

Avvio

```
java -cp out cinemax.CineMax
```

Primo avvio

Al primo avvio, il sistema legge il file data/utenti.csv se presente. In caso contrario, parte senza utenti registrati; il primo utente creato avrà il ruolo di cliente. **Nota:** gli utenti con ruolo PROIEZIONISTA e BIGLIETTAIO devono essere aggiunti manualmente nel CSV oppure assegnati da codice durante lo sviluppo.

4. Ruoli utente e funzionalità per ruolo

Il sistema prevede quattro ruoli:

- GUEST: ospite non registrato.

→ FUNZIONALITA':

- Cercare proiezioni in base a diversi filtri, selezionandone una da visualizzare.
- Scegliere di registrarsi come cliente dal menù iniziale.

- CLIENTE: utente registrato.

→ FUNZIONALITA':

- Cercare proiezioni in base a diversi filtri, selezionandone una da visualizzare;
 - a seguito di questa ricerca può selezionare una proiezione per cui prenotare uno o più biglietti.

- Visualizzare le sue prenotazioni;
- Modificare le sue prenotazioni;
- Cancellare le sue prenotazioni;
- Fare il logout.
- BIGLIETTAIO:
 - FUNZIONALITA':
 - Visualizzare prenotazioni della data odierna;
 - Cercare una prenotazione per:
 - codice prenotazione
 - nome e cognome del cliente
 - titolo del film
 - intervallo di date
 - Fare il logout.
- PROIEZIONISTA:
 - FUNZIONALITA':
 - Aggiungere una nuova proiezione;
 - Modificare una proiezione esistente (*solo se non ha prenotazioni*);
 - Eliminare una proiezione (*solo se non ha prenotazioni*);
 - Logout.

6. Struttura del codice — spiegazione dei package

Il codice è organizzato in livelli, ciascuno con responsabilità specifiche:

Handlers: gestiscono l'interazione con l'utente (input/output)



Managers: eseguono la logica (validazione, operazioni, salvataggio)



Models: contengono i dati (Film, Utente, Proiezione...)

Questa organizzazione è chiamata **architettura a strati**, un pattern molto diffuso nello sviluppo Java.

7. I Modelli (Models)

I modelli sono classi semplici che rappresentano le entità del dominio e includono attributi, costruttori e metodi getter/setter.

Film.java

Rappresenta un film nel catalogo.

Campo	Tipo	Descrizione
titolo	String	Titolo del film
genere	String	Genere (es. "Azione", "Commedia")
regista	String	Nome del regista
anno	int	Anno di produzione
durata	int	Durata in minuti
eta_minima	int	Età minima per la visione

Proiezione.java

Rappresenta una proiezione programmata in sala.

Campo	Tipo	Descrizione
CAPIENZA	int (costante)	200 posti fissi
film	Film	Il film proiettato
dataOra	Date	Data e ora della proiezione
costoBiglietto	Double	Prezzo del biglietto in euro
postiPrenotati	int	Posti già occupati

Metodi importanti:

- `getNumeroPostiDisponibili()`: restituisce $200 - \text{postiPrenotati}$.
- `hasPrenotazioni()`: true se almeno un posto è prenotato.

Prenotazione.java

Rappresenta una prenotazione effettuata da un utente.

Campo	Tipo	Descrizione
codice	String	Codice univoco (es. PR1718123456789)
utente	Utente	L'utente che ha prenotato
proiezione	Proiezione	La proiezione prenotata
numeroBiglietti	int	Numero di biglietti

Metodo importante:

- `getCostoTotale()`: $\text{numeroBiglietti} \times \text{costoBiglietto}$

Il codice viene generato automaticamente con "PR" + `System.currentTimeMillis()`, che lo rende unico.

Utente.java

Rappresenta un utente registrato nel sistema.

Campo	Tipo	Descrizione
nome	String	Nome
cognome	String	Cognome
username	String	Identificativo di login

Campo	Tipo	Descrizione
password	String	Password cifrata (SHA-256)
data_nascita	Date	Data di nascita
domicilio	String	Indirizzo
ruolo	Ruolo	Ruolo nel sistema

Ruolo.java

Enumerazione dei ruoli disponibili: CLIENTE, PROIEZIONISTA, BIGLIETTAIO, GUEST.

8. I Manager (Managers)

I manager contengono le operazioni che si effettuano sui dati contenuti nelle classi models, leggendo e scrivendo sui file CSV.

I manager del progetto sono:

- UtenteManager.java
- ProiezioneManager.java
- PrenotazioneManager.java

UtenteManager.java

Gestisce tutto ciò che riguarda gli utenti.

Responsabilità principali:

- Lettura e scrittura del file CSV degli utenti
- Login con verifica username e password cifrata con SHA-256
- Registrazione di nuovi clienti con validazione dello username univoco
- Mantenimento dello stato dell'utente loggato
- Cifratura delle password tramite algoritmo SHA-256

Di seguito i metodi principali:

caricaUtenti () : legge il file CSV degli utenti e popola la lista utenti in memoria.

→ **Formato CSV atteso:** separatore ; con intestazione

nome;cognome;username;password;data_nascita;domicilio;ruolo

→ **Comportamento:**

- Istanza CSVReader con separatore ; e header presente
 - Per ogni riga del CSV:
 - o Verifica che la riga abbia almeno 7 colonne
 - o Estrae: nome, cognome, username, password già hashata, data nascita, domicilio, ruolo
 - o Converte la stringa della data in oggetto Date usando FORMATO_DATA
 - o Converte la stringa del ruolo nell'enum Ruolo usando Ruolo.valueOf()
 - o Gestisce IllegalArgumentException per ruolo non valido e salta la riga
 - o Crea l'oggetto Utente con i dati estratti
 - Aggiunge alla lista solo gli utenti validi
 - Stampa il numero di utenti caricati correttamente
- **Formato data nascita:** yyyy-MM-dd, per esempio 1990-05-15
- **Formato ruolo:** stringa corrispondente a una costante dell'enum Ruolo, per esempio CLIENTE, BIGLIETTAIO, PROIEZIONISTA
- **Gestione errori:**
- IOException: messaggio di errore su System.err
 - IllegalArgumentException: riga saltata con messaggio di errore specifico
- Nota:** Le password nel file CSV sono già salvate come hash SHA-256, non in chiaro.

salvaUtenti(): persiste tutti gli utenti in memoria nel file CSV, sovrascrivendo il contenuto esistente.

→ **Comportamento:**

- Apre BufferedWriter su FILE_UTENTI
 - Scrive l'intestazione del CSV:
nome;cognome;username;password;data_nascita;domicilio;ruolo
 - Itera su tutti gli utenti in utenti
 - Per ogni utente:
 - o Formatta la data di nascita usando SimpleDateFormat con FORMATO_DATA, se presente
 - o Gestisce data nascita null scrivendo stringa vuota
 - o Costruisce la riga con separatore ;
 - o Scrive il ruolo usando u.getRuolo().name()
 - Chiude il writer
- **Formato output:** nome;cognome;username;hash_sha256;yyyy-MM-dd;domicilio;RUOLO
- **Gestione errori:** IOException gestita con messaggio su System.err
- Nota:** Metodo chiamato automaticamente dopo ogni registrazione.

cifraPassword(String password): cifra una password in chiaro usando l'algoritmo SHA-256.

→ **Parametri:**

- password: password in chiaro da cifrare

→ **Algoritmo SHA-256:**

- Trasforma una stringa in un hash di lunghezza fissa di 64 caratteri esadecimali
- È un algoritmo unidirezionale, quindi non decifrabile
- Lo stesso input produce sempre lo stesso output

→ **Comportamento:**

- Ottiene istanza di MessageDigest per SHA-256
- Converte la password in array di byte
- Calcola l'hash tramite digest()
- Converte ogni byte in due caratteri esadecimali con formato %02x
- Costruisce la stringa esadecimale risultante

→ **Valore di ritorno:** String - hash SHA-256 della password

→ **Esempio:**

- Input: "1234"
- Output: 03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e13f978d7c846f4

→ **Gestione errori:** NoSuchAlgorithmException gestita con messaggio di errore; in tal caso restituisce la password in chiaro

`login(String username, String password)`: effettua il login verificando username e password.

→ **Parametri:**

- username: username dell'utente
- password: password in chiaro inserita dall'utente

→ **Comportamento:**

- Cifra la password inserita chiamando cifraPassword(password)
- Itera su tutti gli utenti nella lista utenti
- Confronta username e password hashata con quelle dell'utente corrente
- Se trova corrispondenza, imposta utenteCorrente = u e restituisce true
- Se nessun utente corrisponde, restituisce false

→ **Validazioni:**

- Username deve esistere nel sistema
- Password, dopo cifratura, deve corrispondere esattamente all'hash salvato

→ **Valore di ritorno:**

- true - credenziali corrette, login effettuato
- false - credenziali errate o utente non trovato

→ **Post-condizioni:** se il login riesce, utenteCorrente punta all'utente autenticato

Nota: La password in chiaro non viene mai salvata o confrontata direttamente; viene sempre cifrata prima del confronto.

`registrazione(String nome, String cognome, String username, String password, String dataNascita, String domicilio)`: registra un nuovo utente con ruolo CLIENTE.

→ **Parametri:**

- nome: nome del nuovo utente
- cognome: cognome del nuovo utente
- username: username scelto, deve essere univoco
- password: password in chiaro, verrà cifrata automaticamente
- dataNascita: data di nascita in formato dd/MM/yyyy, può essere stringa vuota
- domicilio: luogo di domicilio

→ **Validazioni:**

- Username univoco: nessun altro utente deve avere lo stesso username, con confronto case-insensitive

→ **Comportamento:**

- Verifica che lo username non sia già in uso con `equalsIgnoreCase()`
- Se username già esistente, stampa un messaggio e restituisce false
- Converte la data di nascita da stringa a Date usando formato dd/MM/yyyy
- Gestisce `ParseException` per data non valida, lasciando il campo vuoto
- Cifra la password chiamando `cifraPassword(password)`
- Crea nuovo oggetto Utente con ruolo `Ruolo.CLIENTE`
- Aggiunge l'utente alla lista utenti
- Salva la lista aggiornata su CSV con `salvaUtenti()`
- Imposta `utenteCorrente` come il nuovo utente, realizzando il login automatico

→ **Valore di ritorno:**

- true - registrazione avvenuta con successo, utente automaticamente loggato
- false - username già in uso

→ **Formato data atteso:** dd/MM/yyyy, per esempio 15/05/1990

→ **Post-condizioni:**

- Nuovo utente aggiunto a utenti
- File CSV aggiornato
- `utenteCorrente` punta al nuovo utente

→ **Messaggi di errore:**

- "Username già in uso. Scegline un altro."
- "Formato data non valido, il campo verrà lasciato vuoto."

logout() : effettua il logout rimuovendo l'utente corrente dalla sessione.

→ **Comportamento:** imposta utenteCorrente = null

→ **Post-condizioni:** isLoggedIn() restituirà false fino a un successivo login

Nota: Non modifica il file CSV; è un'operazione solo in memoria.

getUtenteCorrente() : restituisce l'utente attualmente loggato.

→ **Valore di ritorno:**

- Utente - l'utente corrente se loggato
- null - se nessun utente è loggato

→ **Utilizzo tipico:** verificare chi è l'utente autenticato prima di operazioni riservate

isLoggedIn(): verifica se un utente è attualmente loggato.

→ **Valore di ritorno:**

- true - c'è un utente loggato, cioè utenteCorrente != null
- false - nessun utente loggato

→ **Utilizzo tipico:** controllo rapido prima di operazioni che richiedono autenticazione

getUtenti(): restituisce la lista completa degli utenti registrati.

→ **Valore di ritorno:** ArrayList<Utente> - lista di tutti gli utenti nel sistema

→ **Utilizzo tipico:**

- Altri manager che necessitano di accedere ai dati utente
- Controlli interni sulle prenotazioni e sulle associazioni utente-prenotazione

Nota: La lista restituita è mutabile; modifiche alla lista restituita influenzeranno lo stato interno del manager.

ProiezioneManager.java

Gestisce il catalogo delle proiezioni.

Responsabilità principali:

- Leggere il file CSV delle proiezioni
- Cercare proiezioni con filtri multi-criterio: titolo, genere, data e prezzo
- Aggiungere nuove proiezioni con validazione delle sovrapposizioni orarie
- Modificare proiezioni esistenti
- Eliminare proiezioni solo se senza prenotazioni attive
- Visualizzare proiezioni singole o raggruppate per titolo
- Ordinare cronologicamente le proiezioni

Metodi principali:

cercaProiezione (CriteriRicercaProiezione criteriRicercaProiezione) : cerca le proiezioni applicando filtri multipli definiti nei criteri di ricerca.

→ **Parametri:**

- criteriRicercaProiezione: oggetto contenente i criteri di ricerca inseriti dall'utente

→ **Comportamento:**

- Utilizza Stream API per filtrare la lista proiezioni
- Applica i filtri in sequenza con AND logico
- Titolo: controlla se il titolo del film contiene la stringa specificata, ignorando maiuscole e minuscole
- Tipologia/Genere: controlla uguaglianza esatta ignorando maiuscole e minuscole
- Periodo data: verifica che la proiezione sia compresa tra dataInizio e dataFine, se indicati
- Prezzo esatto: confronta il costo del biglietto con tolleranza di 0.01
- Gestisce eccezioni restituendo una lista vuota

→ **Valore di ritorno:** List<Proiezione> - lista filtrata delle proiezioni che soddisfano tutti i criteri

raggruppaPerTitolo (List<Proiezione> proiezioni) : raggruppa una lista di proiezioni per titolo del film.

→ **Parametri:**

- proiezioni: lista di proiezioni da raggruppare

→ **Comportamento:**

- Filtra le proiezioni con film e titolo validi
- Utilizza Collectors.groupingBy con chiave uguale al titolo del film

→ **Valore di ritorno:** Map<String, List<Proiezione>> - mappa in cui ogni titolo è associato alla lista delle sue proiezioni

visualizzaRaggruppato (Map<String, List<Proiezione>> raggruppate) : stampa a console le proiezioni raggruppate per titolo.

→ **Parametri:**

- raggruppate: mappa delle proiezioni raggruppate, tipicamente output di raggruppaPerTitolo()

→ **Comportamento:**

- Estrae i titoli in una lista
- Per ogni titolo stampa numero progressivo, titolo e numero di proiezioni
- Mostra un output formattato con intestazione FILM DISPONIBILI

→ **Valore di ritorno:** Nessuno

visualizzaProiezione(Proiezione proiezione) : visualizza i dettagli completi di una singola proiezione in formato leggibile.

→ **Parametri:**

- proiezione: proiezione da visualizzare

→ **Comportamento:**

- Valida che proiezione e film non siano nulli
- Formatta data e ora nel pattern dd/MM/yyyy HH:mm
- Stampa titolo, genere, regista, anno e durata del film
- Stampa data/ora, costo del biglietto e posti liberi

→ **Valore di ritorno:** Nessuno

getProiezioniOrdinate() : restituisce la lista delle proiezioni ordinate cronologicamente per data e ora.

→ **Comportamento:**

- Se la lista proiezioni è null, restituisce una lista vuota
- Filtra elementi nulli e proiezioni con data nulla
- Ordina utilizzando Comparator.comparing(Proiezione::getDataOra)

→ **Valore di ritorno:** List<Proiezione> - lista ordinata in ordine crescente di data e ora

aggiungiProiezione(Proiezione nuovaProiezione) : aggiunge una nuova proiezione al sistema.

→ **Parametri:**

- nuovaProiezione: proiezione da aggiungere

→ **Validazioni:**

- Proiezione, film, data ora e costo non devono essere nulli
- Costo del biglietto maggiore o uguale a 0
- Durata del film maggiore di 0
- Nessuna sovrapposizione con proiezioni esistenti

→ **Comportamento:**

- Inizializza proiezioni come ArrayList se null
- Aggiunge la proiezione alla lista se tutte le validazioni hanno successo

→ **Valore di ritorno:** true se l'aggiunta è riuscita, false in caso di validazione fallita

modificaProiezione(Proiezione proiezioneDaModificare, Film nuovoFilm, Date nuovaDataOra, Double nuovoCostoBiglietto) : modifica i dati di una proiezione esistente.

→ **Parametri:**

- proiezioneDaModificare: proiezione originale da modificare
- nuovoFilm: nuovo film da associare

- nuovaDataOra: nuova data e ora
 - nuovoCostoBiglietto: nuovo costo del biglietto
- **Validazioni:**
- Tutti i parametri devono essere non nulli
 - Nuovo costo maggiore o uguale a 0
 - Nuova durata film maggiore di 0
 - Nessuna prenotazione attiva sulla proiezione
 - Nessuna sovrapposizione con altre proiezioni, escludendo quella in modifica
- **Comportamento:**
- Crea una proiezione candidata con i nuovi dati, mantenendo i posti prenotati
 - Verifica le sovrapposizioni
 - Se la candidata è valida, aggiorna film, data/ora e costo della proiezione originale
- **Valore di ritorno:** true se la modifica è riuscita, false altrimenti

eliminaProiezione(Proiezione proiezioneDaEliminare) : elimina una proiezione dal sistema.

- **Parametri:**
- proiezioneDaEliminare: proiezione da rimuovere
- **Validazioni:**
- Proiezione e lista proiezioni non devono essere nulli
 - La proiezione non deve avere prenotazioni attive
- **Comportamento:**
- Rimuove la proiezione dalla lista utilizzando remove()
- **Valore di ritorno:** true se l'eliminazione è riuscita, false altrimenti

haSovrapposizione(Proiezione nuovaProiezione, Proiezione daEscludere) : verifica se una proiezione si sovrappone temporalmente con altre proiezioni esistenti.

- **Parametri:**
- nuovaProiezione: proiezione da verificare
 - daEscludere: proiezione da escludere dal controllo, tipicamente quella in fase di modifica
- **Comportamento:**
- Calcola l'intervallo orario della nuova proiezione: inizio e fine in millisecondi
 - Itera su tutte le proiezioni esistenti
 - Salta la proiezione da escludere e gli elementi non validi
 - Calcola l'intervallo orario di ciascuna proiezione esistente
 - Verifica se gli intervalli si sovrappongono con la condizione: nuovoInizio < esistenteFine && nuovoFine > esistenteInizio

→ **Valore di ritorno:** true se esiste sovrapposizione, false altrimenti

Regola importante: non possono esistere due proiezioni sovrapposte. L'applicazione verifica che una nuova proiezione inizi solo dopo la fine della precedente, considerando anche la durata del film.

trovaProiezione(String titoloFilm, Date dataOra): cerca una proiezione nella lista per titolo del film e data/ora.

→ **Parametri:**

- titoloFilm: titolo del film
- dataOra: data e ora della proiezione

→ **Comportamento:**

- Itera su tutte le proiezioni presenti nella lista
- Confronta il titolo del film e la data/ora della proiezione
- Restituisce la prima proiezione corrispondente

→ **Valore di ritorno:** Proiezione trovata, oppure null se non esiste

PrenotazioneManager.java

Gestisce le prenotazioni degli utenti.

Responsabilità principali:

- Lettura e scrittura del file CSV delle prenotazioni
- Creazione di nuove prenotazioni con validazione di posti e date
- Visualizzazione delle prenotazioni dell'utente loggato
- Modifica di prenotazioni esistenti con cambio proiezione
- Eliminazione di prenotazioni con rilascio dei posti

caricaPrenotazioni(): legge il file CSV delle prenotazioni e popola la lista in memoria.

→ **Formato CSV atteso:**

```
codice;username;titolo_film;data_ora_proiezione;numero_biglietti
```

→ **Comportamento:**

- Istanza CSVReader con separatore ; e header presente
- Per ogni riga del CSV:
 - o Verifica che la riga abbia almeno 5 colonne
 - o Estrae codice, username, titolo film, data/ora e numero biglietti
 - o Cerca l'utente tramite trovaUtentePerUsername()
 - o Cerca le proiezioni con quel titolo usando proiezioneManager.cercaProiezione()
 - o Filtra la proiezione corretta confrontando la data/ora
 - o Converte il numero biglietti da stringa a intero
 - o Crea l'oggetto Prenotazione e imposta il codice letto dal file

- Aggiunge alla lista solo le prenotazioni valide
- Stampa il numero di prenotazioni caricate

→ **Gestione errori:**

- IOException: se il file non esiste, stampa un messaggio informativo
- NumberFormatException: se il numero biglietti non è valido, salta la riga

salvaPrenotazioni() : persiste tutte le prenotazioni in memoria nel file CSV, sovrascrivendo il contenuto esistente.

→ **Comportamento:**

- Apre BufferedWriter su FILE_PRENOTAZIONI
- Scrive l'intestazione del CSV:
codice;username;titolo_film;data_ora_proiezione;numero_biglietti
- Itera su tutte le prenotazioni in prenotazioni
- Per ogni prenotazione scrive una riga formattata con separatore ;
- Formatta la data/ora usando SimpleDateFormat con FORMATO_DATA
- Chiude il writer

→ **Gestione errori:** IOException gestita con messaggio su System.err

Nota: nel progetto il metodo viene chiamato al logout del cliente, dopo le operazioni sulle prenotazioni.

trovaUtentePerUsername(String username) : cerca un utente nella lista degli utenti per username.

→ **Parametri:**

- username: username da cercare

→ **Comportamento:**

- Itera su tutti gli utenti restituiti da userManager.getUtenti()
- Confronta gli username usando equals()

→ **Valore di ritorno:**

- Utente - utente trovato
- null - se nessun utente corrisponde allo username

creaPrenotazione(Proiezione proiezione, int numeroBiglietti) : crea una nuova prenotazione per l'utente attualmente loggato.

→ **Parametri:**

- proiezione: proiezione da prenotare
- numeroBiglietti: numero di biglietti richiesti

→ **Validazioni:**

- Deve esserci un utente loggato
- La proiezione deve avere abbastanza posti disponibili

- La data della proiezione deve essere futura

→ **Comportamento:**

- Recupera l'utente corrente da userManager
- Verifica tutte le condizioni di validità
- Crea un nuovo oggetto Prenotazione con codice generato automaticamente
- Aggiorna i posti prenotati nella proiezione: postiPrenotati += numeroBiglietti
- Aggiunge la prenotazione alla lista prenotazioni
- Stampa il codice della prenotazione creata

→ **Valore di ritorno:**

- true - se la prenotazione è stata creata con successo
- false - se una validazione fallisce

Nota: le modifiche vengono tenute in memoria; il salvataggio su file avviene chiamando salvaPrenotazioni().

visualizzaPrenotazioni() : restituisce la lista delle prenotazioni dell'utente attualmente loggato.

→ **Comportamento:**

- Recupera l'utente corrente da userManager
- Itera su tutte le prenotazioni presenti nella lista prenotazioni
- Filtra quelle in cui prenotazione.getUtente().getUsername() corrisponde allo username dell'utente corrente

→ **Valore di ritorno:** ArrayList<Prenotazione> - lista delle prenotazioni dell'utente corrente, anche vuota

modificaPrenotazione(String codice, Proiezione nuovaProiezione) : modifica una prenotazione esistente cambiando la proiezione associata.

→ **Parametri:**

- codice: codice della prenotazione da modificare
- nuovaProiezione: nuova proiezione scelta

→ **Validazioni:**

- La prenotazione deve esistere
- La vecchia proiezione deve essere futura
- La nuova proiezione deve essere futura
- La nuova proiezione deve avere posti disponibili sufficienti

→ **Comportamento:**

- Cerca la prenotazione per codice
- Rilascia i posti dalla vecchia proiezione: postiPrenotati -= numeroBiglietti
- Occupa i posti nella nuova proiezione: postiPrenotati += numeroBiglietti
- Aggiorna il riferimento alla proiezione nella prenotazione

- Stampa un messaggio di conferma

→ **Valore di ritorno:**

- true - se la modifica è andata a buon fine
- false - se una validazione fallisce

Nota: le modifiche vengono tenute in memoria; il salvataggio su file avviene chiamando `salvaPrenotazioni()`.

`eliminaPrenotazione(String codice)` : elimina una prenotazione esistente dal sistema.

→ **Parametri:**

- codice: codice della prenotazione da eliminare

→ **Validazioni:**

- La prenotazione deve esistere
- La data della proiezione deve essere futura

→ **Comportamento:**

- Cerca la prenotazione per codice
- Verifica che la proiezione sia futura
- Rilascia i posti dalla proiezione: `postiPrenotati -= numeroBiglietti`
- Rimuove la prenotazione dalla lista prenotazioni
- Stampa un messaggio di conferma

→ **Valore di ritorno:**

- true - se l'eliminazione è andata a buon fine
- false - se una validazione fallisce

→ **Messaggi di errore:**

- "Prenotazione non trovata."
- "Non puoi cancellare una prenotazione per una proiezione già passata."

`trovaPerCodice(String codice)` : cerca una prenotazione nella lista per codice univoco.

→ **Parametri:**

- codice: codice della prenotazione da cercare

→ **Comportamento:**

- Itera su tutte le prenotazioni nella lista prenotazioni
- Confronta i codici usando `equals()`

→ **Valore di ritorno:**

- Prenotazione - la prenotazione trovata
- null - se nessuna prenotazione corrisponde al codice

`getTutteLePrenotazioni()` : restituisce tutte le prenotazioni presenti nel sistema.

- **Comportamento:** restituisce il riferimento alla lista interna prenotazioni
- **Valore di ritorno:** ArrayList<Prenotazione> - lista completa di tutte le prenotazioni
- **Utilizzo tipico:** usato dal bigliettaio per visualizzare le prenotazioni del giorno e per cercare prenotazioni

9. Gli Handler (Handlers)

Gli handler gestiscono i menu interattivi: leggono l'input dell'utente e richiamano i metodi dei Manager.

AuthHandler.java

Gestisce login, registrazione e reindirizzamento ai menu in base al ruolo.

- **gestisciLogin:** richiede username e password → chiama `utenteManager.login()` → se successo, apre il menu del ruolo corretto tramite `visualizzaMenu()` → al ritorno esegue `utenteManager.logout()`
- **gestisciRegistrazione:** raccoglie nome, cognome, username, password, data nascita, domicilio → chiama `utenteManager.registrazione()` (crea utente con ruolo CLIENTE) → apre automaticamente il menu cliente → al ritorno esegue il logout
- Reindirizzamento ruoli: **visualizzaMenu()** legge il ruolo dell'utente corrente e chiama il `mostraMenu()` del handler corrispondente (CLIENTE, BIGLIETTAIO, PROIEZIONISTA)

GuestMenuHandler.java

Gestisce il menu per gli utenti non autenticati (guest).

- **mostraMenu:** ciclo continuo con opzioni "Cerca proiezioni" (1) e "Torna al menu principale" (0)
- **gestisciRicercaProiezioni:** acquisisce criteri di ricerca tramite `RicercaProiezioneConsoleHelper.acquisisciCriteriRicerca()` → chiama `proiezioneManager.cercaProiezione()` → mostra risultati tramite helper
- **selezionaEVisualizzaProiezione:** permette di selezionare una proiezione dalla lista dei risultati → chiama `proiezioneManager.visualizzaProiezione()` per mostrare i dettagli completi

ClienteMenuHandler.java

Gestisce il menu per gli utenti clienti autenticati.

- **mostraMenu**: ciclo continuo con opzioni: Cerca e prenota (1), Visualizza prenotazioni (2), Modifica prenotazione (3), Cancella prenotazione (4), Logout (0)
- **cercaEPrenota**: acquisisce criteri di ricerca tramite RicercaProiezioneConsoleHelper → chiama proiezioneManager.cercaProiezione() → mostra risultati → utente seleziona una proiezione → richiede numero biglietti → chiama prenotazioneManager.creaPrenotazione() (controlla automaticamente posti disponibili e data futura)
- **visualizzaPrenotazioni**: chiama prenotazioneManager.visualizzaPrenotazioni() (filtra automaticamente per utente corrente) → stampa la lista formattata con codice, film, data, biglietti e totale
- **modificaPrenotazione**: mostra le prenotazioni dell'utente → utente seleziona quale modificare → cerca nuova proiezione con criteri → seleziona nuova proiezione → chiama prenotazioneManager.modificaPrenotazione() (verifica che vecchia e nuova data siano future)
- **cancellaPrenotazione**: mostra le prenotazioni dell'utente → utente seleziona quale cancellare → richiede conferma → chiama prenotazioneManager.eliminaPrenotazione() (verifica che la data sia futura)
- **logout**: chiama prenotazioneManager.salvaPrenotazioni() per persistenza finale → esce dal menu

BigliettaioMenuHandler.java

Gestisce il menu per gli utenti con ruolo BIGLIETTAIO.

- **mostraMenu**: ciclo continuo con opzioni: Visualizza prenotazioni di oggi (1), Cerca prenotazione (2), Logout (0)
- **visualizzaPrenotazioniOggi**: recupera tutte le prenotazioni con prenotazioneManager.getTutteLePrenotazioni() → filtra quelle con data proiezione uguale alla data odierna → stampa la lista formattata
- **cercaPrenotazione**: sottomenu con 4 criteri di ricerca:
 - **cercaPerCodice**: cerca corrispondenza esatta con getCode().equals(codice)
 - **cercaPerNomeCognome**: ricerca case-insensitive con contains() su nome e cognome del cliente

- **cercaPerTitolo**: ricerca parziale case-insensitive su `getFilm().getTitolo()`
- **cercaPerDate**: filtra prenotazioni con data proiezione compresa tra data inizio e data fine (entrambe opzionali, formato `dd/MM/yyyy`)
- **stampaListaPrenotazioni**: mostra per ogni prenotazione: codice, cliente, film, data (`dd/MM/yyyy HH:mm`), numero biglietti, totale

ProiezionistaMenuHandler.java

Gestisce il menu per gli utenti con ruolo PROIEZIONISTA.

- **mostraMenu**: ciclo continuo con opzioni: Aggiungi proiezione (1), Modifica proiezione (2), Elimina proiezione (3), Logout (4)
- **aggiungiProiezione**: acquisisce dati film (titolo, genere, regista, anno, durata, età minima), data/ora (`dd/MM/yyyy HH:mm`) e costo biglietto → chiama `proiezioneManager.aggiungiProiezione()` (verifica automaticamente assenza sovrapposizioni orarie)
- **modificaProiezione**: seleziona proiezione da lista (con filtro per titolo) → verifica che non abbia prenotazioni → acquisisce nuovi dati (valori correnti proposti come default) → chiama `proiezioneManager.modificaProiezione()` (verifica assenza prenotazioni e sovrapposizioni)
- **eliminaProiezione**: seleziona proiezione da lista (con filtro per titolo) → chiama `proiezioneManager.eliminaProiezione()` (consentita solo se senza prenotazioni)
- **Metodi helper per input utente**: gestiscono acquisizione dati con validazione e valori di default:
 - `leggiStringaObbligatoria()`: richiede testo, mostra valore corrente tra parentesi se presente
 - `leggiInteroPositivo()`: richiede numero intero > 0
 - `leggiDataOra()`: richiede data/ora in formato `dd/MM/yyyy HH:mm`
 - `leggiDoublePositivo()`: richiede numero decimale ≥ 0 (accetta virgola o punto)
 - `acquisisciDatiFilm()`: richiede tutti i campi di un film
 - `selezionaProiezioneDaLista()`: mostra proiezioni ordinate, permette filtro per titolo, restituisce quella selezionata

Regole di business (applicate da `ProiezioneManager` e comunicate all'utente in caso di errore):

- Aggiunta: la nuova proiezione non deve sovrapporsi ad altre esistenti
- Modifica: la proiezione non deve avere prenotazioni attive e non deve sovrapporsi ad altre
- Eliminazione: la proiezione non deve avere prenotazioni attive

10. Gli Helper (Helpers)

Classi di supporto riutilizzabili.

CSVReader.java

Classe di utilità per leggere file CSV e convertire ogni riga in oggetti Java tramite una funzione mapper.

- **Metodo principale:** `readFile(String percorsoFile, Function<List<String>, T> mapper)`: apre il file, salta l'intestazione se presente, divide ogni riga in campi (gestendo virgolette annidate), applica la funzione mapper per convertire i valori in oggetto, restituisce lista di oggetti
- **Metodi di utilità per parsing** (statici, restituiscono null in caso di errore):
 - `parseInt(String valore)`: converte stringa in Integer
 - `parseDouble(String valore)`: converte stringa in Double
 - `parseDate(String valore, String formato)`: converte stringa in Date usando il formato specificato (es. "yyyy-MM-dd HH:mm:ss")
- **Gestione CSV:** supporta campi tra virgolette doppie ("...") e virgolette doppie escape ("" dentro virgolette)

ColoreConsole.java

Fornisce output colorato nel terminale tramite **codici ANSI**.

Metodo	Colore	Uso tipico
<code>successo(testo)</code>	Verde	Operazione riuscita

Metodo	Colore	Uso tipico
errore(testo)	Rosso	Errore o blocco
avvertimento(testo)	Giallo	Avvisi
header(testo)	Ciano	Titoli di sezione
titoloFilm(testo)	Viola	Nomi dei film
benvenuto(testo)	Verde	Messaggio di benvenuto

Nota: i colori ANSI sono supportati su Linux, macOS e PowerShell/Windows Terminal, ma potrebbero non funzionare nel vecchio cmd.exe di Windows.

RicercaProiezioneConsoleHelper.java

Classe di utilità per gestire l'interazione a console durante la ricerca delle proiezioni.

- **Flusso principale di ricerca:**
 - `acquisisciCriteriRicerca(Scanner scanner)`: raccoglie dall'utente titolo (parziale), genere, filtro date (nessuno/dopo una data/tra due date), filtro costo (nessuno/minore o uguale/tra due valori/uguale) → restituisce oggetto `CriteriRicercaProiezione`
 - `mostraRisultatiRicerca(List<Proiezione> risultati)`: stampa i risultati raggruppati per titolo, con genere, data/ora (dd/MM/yyyy HH:mm) e prezzo di ogni proiezione
 - `selezionaProiezione(Scanner scanner, List<Proiezione> risultati, String prompt)`: permette all'utente di scegliere un numero dalla lista, restituisce la Proiezione selezionata (o null se sceglie 0)
- **Metodi di supporto:**
 - `leggiData()`: richiede data in formato dd/MM/yyyy, restituisce `Date` (o null se input vuoto)
 - `leggiDouble()`: richiede numero decimale (accetta virgola o punto), restituisce `Double` (o null se input vuoto)

- `stampaHeader()` : stampa intestazione colorata
- `formattaErrore()` : restituisce messaggio di errore colorato

11. I dati — file CSV

I dati vengono salvati in file CSV all'interno della cartella `data/`.

`data/utenti.csv`

Ogni riga rappresenta un utente. Colonne attese:

nome,cognome,username,password,data_nascita,domicilio,ruolo

- password è salvata come hash SHA-256 (mai in chiaro)
- data_nascita è in formato dd/MM/yyyy
- ruolo è uno dei valori: CLIENTE, BIGLIETTAIO, PROIEZIONISTA

Esempio:

Mario,Rossi,mrossi,5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a...,01/01/1990
,Roma,CLIENTE

`data/proiezioni.csv`

Ogni riga rappresenta una proiezione programmata.

titolo,genere,regista,anno,durata,eta_minima,dataOra,costoBiglietto,postiPrenotati

- dataOra è in formato dd/MM/yyyy HH:mm
- costoBiglietto usa il punto come separatore decimale

Esempio:

Inception,Sci-Fi,Christopher Nolan,2010,148,14,15/07/2026 21:00,9.50,45

`data/prenotazioni.csv` (generato automaticamente)

Viene creato e aggiornato dal `PrenotazioneManager`.

codice,username,titolo_film,dataOra_proiezione,numeroBiglietti

12. Sicurezza

Password

Le password non vengono mai salvate in chiaro. Vengono convertite in **hash SHA-256** prima di essere memorizzate nel CSV:

// Esempio di cifratura

```
String hashPassword = UtenteManager.cifraPassword("miaPassword123");
```

// Risultato: una stringa esadecimale di 64 caratteri

Al momento del login, la password inserita viene anch'essa cifrata e confrontata con quella salvata.

Validazioni implementate

- Username univoco in fase di registrazione
- Controllo che l'utente sia loggato prima di prenotare
- Controllo posti disponibili prima di creare una prenotazione
- Blocco modifica/eliminazione proiezioni con prenotazioni attive
- Blocco cancellazione di prenotazioni per proiezioni già passate